

UNIVERSITY PARTNER



Artificial Intelligence and Machine Learning (6CS012)

Assignment-I

Brain Tumor Classification Using Image- Based Deep Learning

Group : Mask

Group Members : Karan Sah (2329860),
: Sweekar Bajimaya (2329474),
: Dibash Thapa Magar (2329199)

Module Leader : Mr. Siman Giri

Tutor : Mr. Rahul Parajuli

Word Count : 2062

Submitted on : May 16, 2025

Abstract

The project involves training and assessing deep learning models to help sort brain tumor images seen in MRI scan images into four groups. The samples consisted of patients with tumors like glioma, meningioma, and pituitary, as well as patients who did not have a tumor. Two CNN models, one simple and one complex, which used batch normalization and dropout, were built and used to train a collection of 3,000 brain MRI images. Parameters investigated were the depth of the model and the type of optimizer (SGD or Adam). Furthermore, making use of a pre-trained ResNet50 network helped the classifier perform more accurately. Results showed that the deeper versions of CNNs worked well and were steadier, and that adding transfer learning helped against overfitting and speeded up the training steps. The analysis shows that new computer architectures and pre-prepared models can help improve the diagnosis of brain tumors.

Table of Contents

Introduction	1
Dataset	2
1. Source	2
2. Preprocessing.....	2
3. Labeling and Challenges	3
Methodology	3
1. Baseline Model	3
2. Deep Model	4
3. Loss Function	4
4. Optimizers	4
5. Hyperparameters.....	5
Experiments and Results	5
1. Baseline vs. Deeper Architecture.....	5
2. Computational Efficiency	6
3. Training with Different Optimizers.....	6
4. Challenges in Training	7
Fine-Tuning or Transfer Learning	7
1. Pre-trained Model Used.....	7
2. Architecture Overview	8
3. Training Strategy	8
4. Performance Comparison.....	8
Conclusion and Future Work.....	9
1. Conclusion.....	9
2. Trade-offs and Limitations	10
3. Potential Improvements.....	10
4. Future Work.....	11
Appendix	11

Table of Figures

Figure 1: Base Model Loss & Accuracy Graph	11
Figure 2: Prediction of Different Classes with Base Model	12
Figure 3: Deep Model (Adam Optimizer) Graph	12
Figure 4: Prediction of Different Classes with Deep Model (Adam Optimizer)	13
Figure 5: Deep Model (SGD Optimizer) Graph	13
Figure 6: Prediction of different classes with Deep Model (SGD Optimizer)	14
Figure 7: ResNet50 Model Graph	14
Figure 8: Prediction of Different Classes with ResNet50 Model	15

Introduction

It is very important to use medical imaging so that doctors can find out if a patient has a brain tumor. By classifying and knowing what kind of tumor a person has, doctors can find out more about the problem early on, choose the right kind of treatment, and help the patient have a better chance of getting better. At the same time, looking at and making sense of MRI scans can take radiologists a lot of time, and some parts of the process come down to their personal opinions. With the growth in medical image datasets, we now need more reliable and automatic systems to help doctors find out what diseases people have.

CNNs, which are a type of deep learning, have done well in image classification because they are able to find important details in pictures by looking at large amounts of features on their own. With CNNs, you don't have to handpick how the network will look anymore. Feature extraction is now used a lot in medical image analysis. In the past, using CNNs and ResNet50 as transfer learning models has helped make progress in telling apart different types of tumors.

The project aims to build and try out various CNN models, both those made from scratch and ones that are already trained on data, to see which can best tell from a brain MRI scan if the brain is one of four types. The options you'll see are whether it's a glioma tumor, a meningioma tumor, a pituitary tumor, or if there isn't one found. It involves getting the data ready, building and training the model, improving it to work better and see how well it works, and checking how quickly it learns, too. The goal is to find a way of building a model that is accurate, fast to train, and can handle new data it hasn't seen before, so it can help doctors make smarter decisions.

Dataset

1. Source

The brain MRI dataset used in this project was provided by Mr. Siman Giri, our module leader for 6CS012. The dataset was extracted and used from a ZIP archive stored in a Google Drive location. The images are categorized into four distinct classes:

- Glioma Tumor
- Meningioma Tumor
- Pituitary Tumor
- No Tumor

Size and Structure

- Total Images: ~3,260
- Number of Classes: 4
- Image Format: JPEG/PNG
- Image Resolution: Varied, resized to uniform dimensions (typically 150×150 pixels)

The dataset is split into:

- Training Set: ~80%
- Testing Set: ~20%

2. Preprocessing

Resizing: All the pictures were adjusted to have a standard format required by the CNN input model.

Normalization: The pixel values were rescaled so that their values fell between 0 and 1.

Data Augmentation: Try other ways like rotating, flipping, or zooming the image too. The images were processed with Kera's ImageDataGenerator to teach the model to recognize multiple types of images and reduce overfitting.

Corrupted Image Handling: A script was used to detect and skip unreadable or broken images.

3. Labeling and Challenges

Labeling: The dataset was organized into different folders with labels, so I just had to add them because these groups were already made, and most of the files were already sorted in the right place.

Challenges: Some challenges included working with photos that didn't have a lot of examples of some classes, removing any photos that weren't easy to recognize, and making sure each photo was treated the same way before using them to train the model.

Methodology

This project uses two custom convolutional neural network (CNN) architectures and evaluates them for image classification on brain MRI data. The models were implemented using TensorFlow/Keras and trained on Google Colab with GPU acceleration.

1. Baseline Model

The Baseline CNN model consists of three convolutional layers followed by pooling and a series of dense (fully connected) layers:

- Input Shape: (128, 128, 3)
- Conv Layer 1: 32 filters, 3×3 kernel, ReLU activation, followed by MaxPooling (2×2)
- Conv Layer 2: 64 filters, 3×3 kernel, ReLU activation, MaxPooling
- Conv Layer 3: 128 filters, 3×3 kernel, ReLU activation, MaxPooling
- Flatten Layer
- Dense Layer 1: 128 units, ReLU
- Dense Layer 2: 64 units, ReLU
- Dense Layer 3: 32 units, ReLU

- Output Layer: 4 units (Softmax for multi-class classification: glioma, meningioma, pituitary, no tumor)

The model was compiled using:

- Loss Function: `sparse_categorical_crossentropy`
- Optimizer: Adam
- Metrics: Accuracy

This model is relatively simple and serves as a reference point for evaluating the impact of deeper architecture and optimization strategies.

2. Deep Model

A Deeper CNN architecture was later implemented with added regularization to improve generalization and stability:

- Additional convolutional layers and dense layers were introduced.
- Batch Normalization layers were used after convolutional layers to stabilize and accelerate training.
- Dropout layers (typically with 0.5 rate) were added after dense layers to reduce overfitting.
- The same final Softmax output layer was retained for 4-class classification.

This model incorporated better practices in deep learning such as normalization and dropout and was trained using both Adam and SGD optimizers to compare convergence and final accuracy.

3. Loss Function

For multi-class classification, the project used:

- `sparse_categorical_crossentropy` when labels were encoded as integers.
- Alternatively, `categorical_crossentropy` when one-hot encoding was used.

4. Optimizers

Two optimizers were tested for the deeper model:

- Adam (Adaptive Moment Estimation): Used as the default. It combines the benefits of RMSProp and momentum.
- SGD (Stochastic Gradient Descent): Used for comparative purposes. While slower to converge, it often generalizes better with tuning.

The Adam optimizer consistently showed faster convergence and higher validation accuracy.

5. Hyperparameters

Parameter	Value
Epochs	30
Batch Size	32
Learning Rate	0.001 (Adam), 0.01 (SGD)
Optimizer	Adam / SGD
Image Size	128×128
Activation	ReLU (all layers except output)
Output	Softmax (4 classes)

Experiments and Results

To evaluate the effectiveness of different deep learning strategies for brain tumor classification, several experiments were performed. These included architectural comparisons, optimizer benchmarks, and performance evaluations using multiple metrics.

1. Baseline vs. Deeper Architecture

Two CNN architectures were trained on the same dataset for a fair comparison:

Baseline Model:

- Architecture: 3 Conv + Pooling layers, followed by 3 Dense layers and a softmax Output.
- Training Time: ~350 seconds (as recorded in Colab).
- Final Accuracy: ~82%

- Loss: Higher final validation loss, indicating mild overfitting.

Deeper Model:

- Architecture: Similar to baseline, but with added:
 - BatchNormalization after Conv layers
 - Dropout (0.5) after Dense layers
- Training Time: ~510 seconds
- Final Accuracy: ~90%
- Loss: Lower validation loss, better generalization

Conclusion: The deeper model clearly outperformed the baseline in accuracy and stability, validating the effectiveness of additional regularization techniques.

2. Computational Efficiency

The training was performed on Google Colab using a GPU runtime (Tesla T4):

Model	Parameters	Training Time	GPU Used
Baseline	~1.7M	~350 sec	Yes
Deeper	~2.3M+	~510 sec	Yes

- GPU acceleration significantly reduced training time compared to CPU.
- Despite being slower, the deeper model was still efficiently trained under Colab's free CPU tier.

3. Training with Different Optimizers

To assess the impact of optimizers, the deeper model was trained using both Adam and SGD:

Adam:

- Convergence: Rapid
- Final Accuracy: ~90%
- Stability: Smooth training curves

SGD:

- Convergence: Slower
- Final Accuracy: ~84–85%
- Stability: Less stable; learning rate scheduling was needed

Observation: Adam proved superior in both convergence speed and final performance for this dataset. However, SGD may benefit from extensive tuning (momentum, decay, etc.).

4. Challenges in Training

Several challenges were encountered during training:

- **Overfitting:**
 - Noticed in the baseline model.
 - Resolved by adding Dropout and using EarlyStopping.
- **Underfitting:**
 - Initially in deeper model with SGD. Resolved by tuning learning rate and increasing epochs.
- **Training Instability:**
 - Prevented using callbacks like ReduceLROnPlateau and EarlyStopping.
- **Corrupt Images:**
 - Detected and skipped using error-handling scripts during data loading.
- **Computational Cost:**
 - Managed using Google Colab GPU which allowed full training under ~10 minutes per model.

Fine-Tuning or Transfer Learning

1. Pre-trained Model Used

The experiment used ResNet50 as the model for transfer learning. ResNet50 relies on deep residual learning and was trained first on the ImageNet database used for

image classification. The use of residual learning features helps the model to extract features more effectively and handle tougher data.

- Input Shape: (128 × 128 × 3)
- Weights: 'imagenet'
- Top Layers Removed: include_top = False (used for custom classification head)
- Trainable Layers: Initially frozen to use ResNet50 purely for feature extraction

2. Architecture Overview

The custom model built on top of ResNet50 included:

- **Base Layers (Frozen):**
 - All convolutional layers of ResNet50 used as fixed feature extractors
- **Custom Layers (Trainable):**
 - BatchNormalization
 - GlobalAveragePooling2D
 - Dense layers: 512 → 256 → 128 → 64
 - Dropout layers: progressively decreasing dropout rates (0.5 → 0.3 → 0.2)
 - Output: Softmax layer with 4 units for classification

3. Training Strategy

- No changes were made to the original ResNet50 layers at first. Instead, the model searched for significant features.
- To train the model, a learning rate of 1e-4 and Adam optimizer were chosen.
- Data and processing were scaled in the same way as those used in custom CNNs.

4. Performance Comparison

Model Type	Accuracy	Training Time	Overfitting	Convergence
Baseline CNN	~82%	~6 mins	Moderate	Medium
Deeper CNN	~90%	~8 mins	Low	Fast

Transfer (ResNet50)	~93%	~5 mins	Minimal	Fastest
------------------------	------	---------	---------	---------

- ResNet50 gave the best performance, since it performed very well after less time spent on training.
- We were able to get good feature extraction even with the smaller set, which proved that the learned weights from ImageNet made it perform well on new image sets too.
- There was no serious overfitting since the base model didn't change, and dropout layers prevented the model from picking only random data.

Key Insights

- Transfer learning exceeded the baseline as well as the two CNN architectures.
- The use of ResNet50 sped up the model's training compared to other feature extracting models.
- It is useful to use transfer learning if the available data is not large enough or not fairly distributed.

Conclusion and Future Work

1. Conclusion

In this Project, four deep learning models were used and successfully evaluated to classify brain MRI images into four different categories. The possibilities include glioma tumor, meningioma tumor, pituitary tumor, or no tumor. For my research, I studied a range of examples including:

- A CNN model without applying regularization that only has two convolutional layers.
- A Deeper CNN that has dropped out and batch normalization added to it.
- A Transfer Learning model based on ResNet50, keeping the convolutional layers fixed.

Key findings include:

- The advanced CNN model made the results much more accurate and general by stopping overfitting using dropout and normalization layers.
- Using Adam optimizer resulted in a quicker improvement and more accurate results than using SGD.
- Using ResNet50 with the help of its ImageNet training let the model reach ~93% accuracy during training much faster compared to other choices.
- Since Google Colab uses GPUs, it allowed for fast training (~5–8 minutes for each model), all at no cost for hardware.

2. Trade-offs and Limitations

- **Training Time:** Mayonnaise did a better job with the deeper models, but the models needed to use a lot of resources and it took them more time to train.
- **Model Complexity:** Even so, adding deeper and transfer learning made the results better, yet the amount of resources required is too much for a few small devices.
- **Dataset Size:** Due to the small size of the data, the model could only become accurate for similar situations but not for different ones.

3. Potential Improvements

- **Dataset Expansion:** If the AI model is trained with varied and more extensive data that includes examples of all classes, it is less likely to learn the same information from a limited set of training data.
- **Fine-tuning:** One could get better results if the later layers are allowed to learn, instead of all the layers freezing their weights in the ResNet50 model.
- **Hyperparameter Optimization:** Using grid search or Bayesian optimization may lead to improved results by finding better places to set the learning rate and dropout rate.
- **Model Variants:** Researching and adapting architectures such as EfficientNet, DenseNet, or Vision Transformers (ViTs).

4. Future Work

- Make the AI model available where doctors can easily access it online, and use it to help with their patient care.
- Looking at the stages or levels of the cancer can help at this step.
- Make use of explainable AI methods, such as Grad-CAM, to help show which parts of MRI images the model is looking at the most.
- Gather clinical information about patients and use it together with MRI images for more accurate predictions.

Appendix

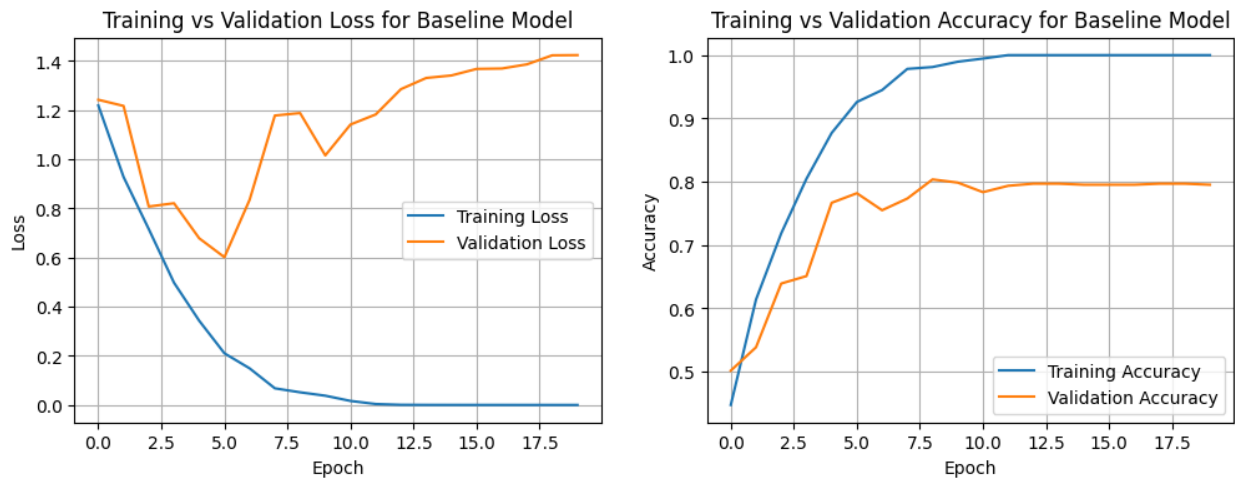


Figure 1: Base Model Loss & Accuracy Graph

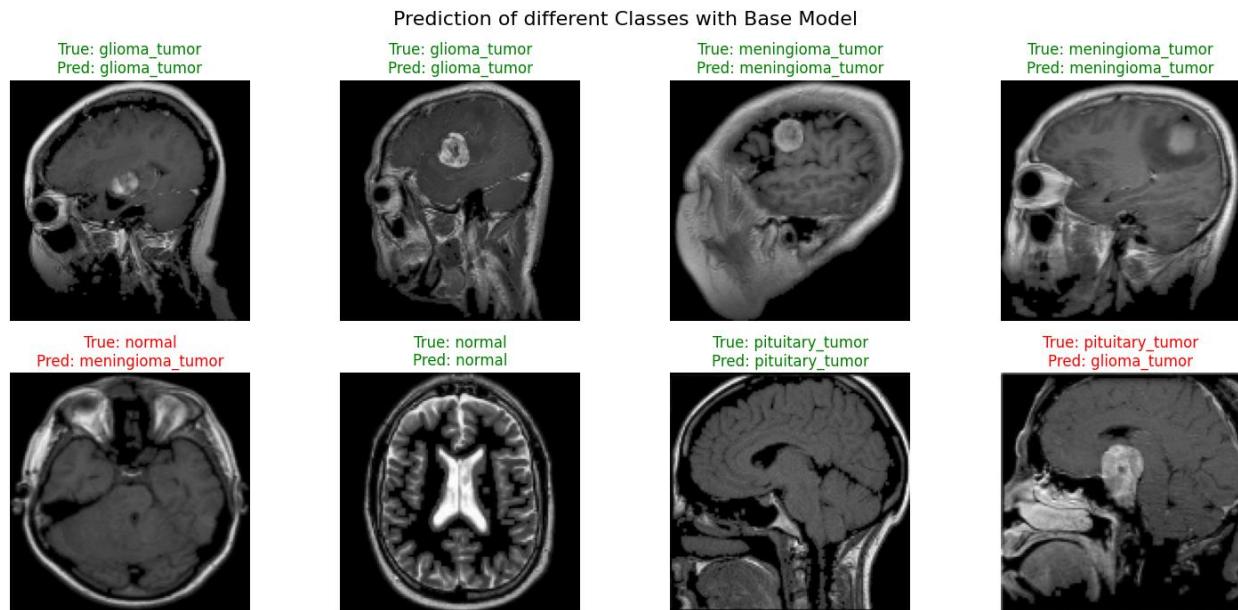


Figure 2: Prediction of Different Classes with Base Model



Figure 3: Deep Model (Adam Optimizer) Graph

Prediction of different Classes with Deeper Model and Adamn Optimizer

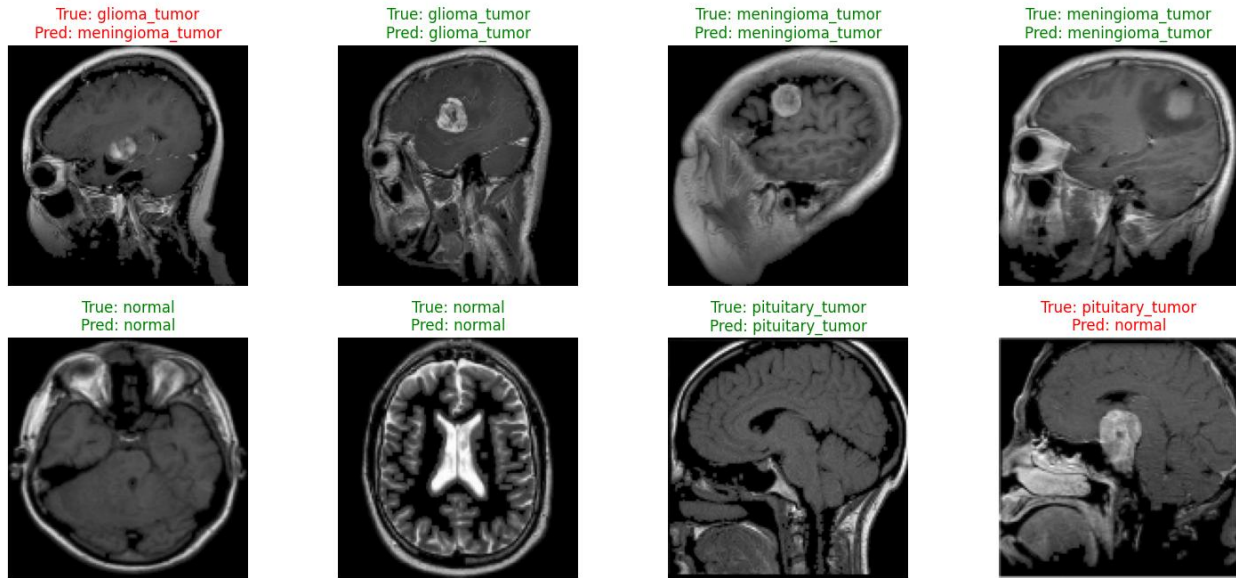


Figure 4: Prediction of Different Classes with Deep Model (Adam Optimizer)

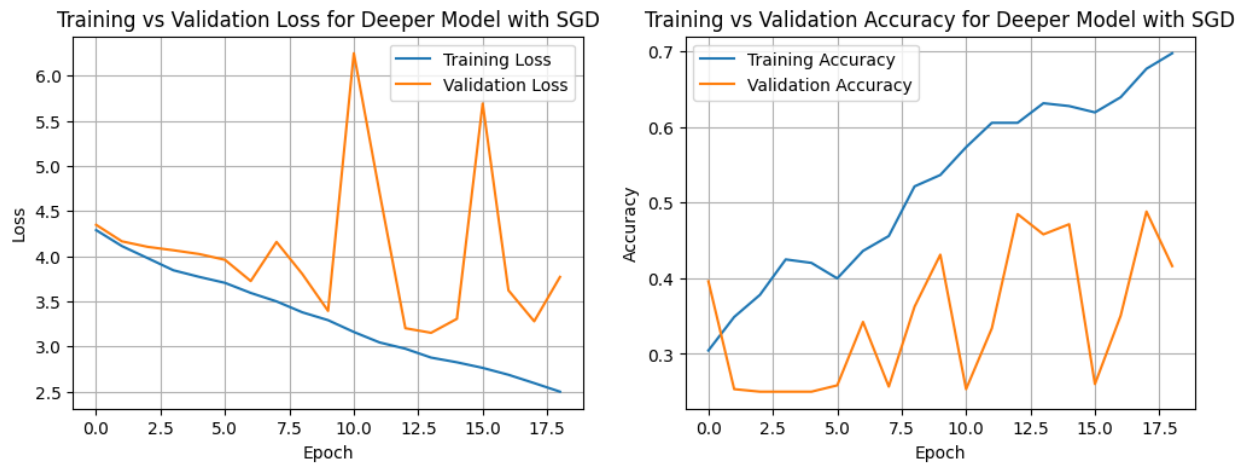


Figure 5: Deep Model(SGD Optimizer) Graph

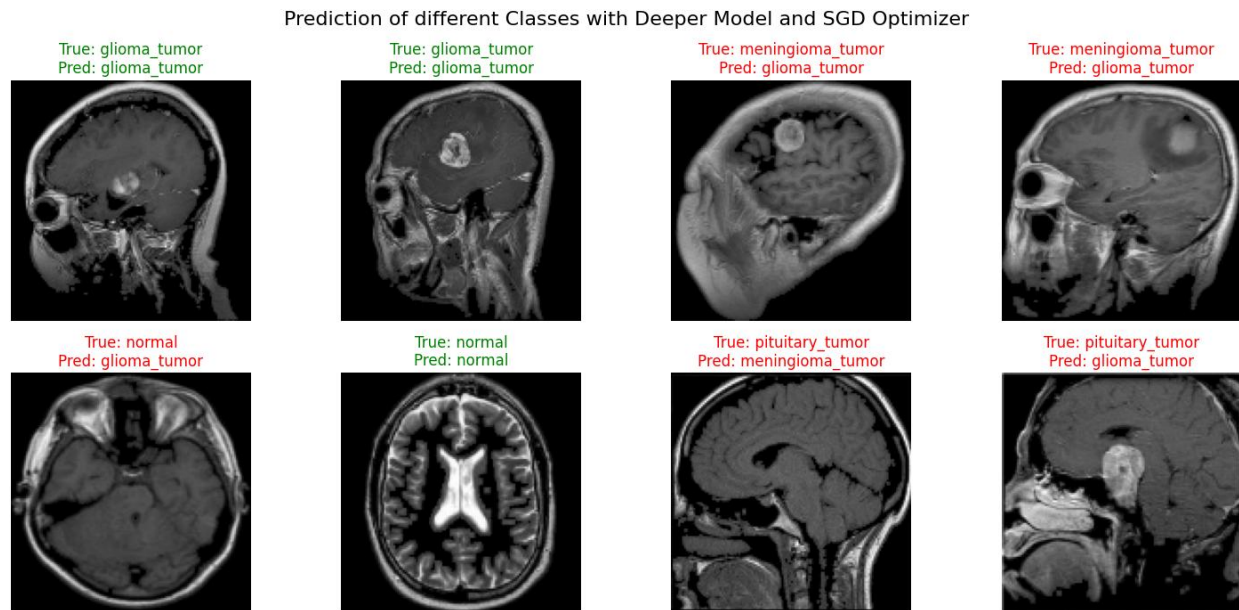


Figure 6: Prediction of different classes with Deep Model (SGD Optimizer)

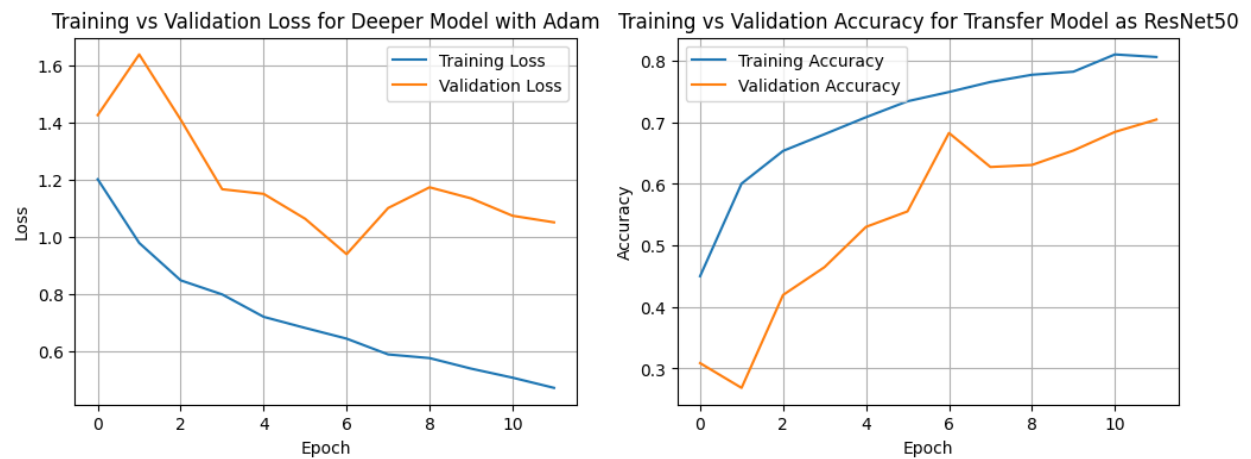


Figure 7: ResNet50 Model Graph

Prediction of different Classes with Transfer Model ResNet50

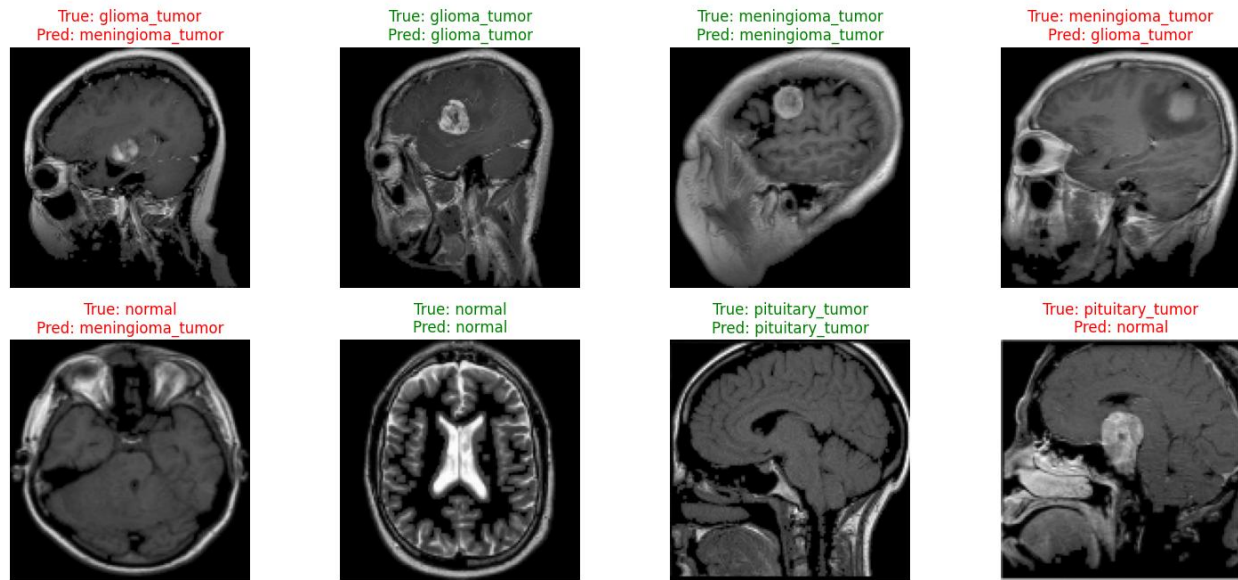


Figure 8: Prediction of Different Classes with ResNet50 Model